1. Project Planning & Management

1.1 Project Proposal

Project Title

Multimodal Medical RAG System & Specialist Recommendation Engine

1. Project Overview

The modern healthcare experience is often fragmented. Patients struggle to understand complex medical literature, decode their symptoms, and find the appropriate specialists for their specific conditions. This project aims to bridge that gap by developing an end-to-end, AI-powered Multimodal Medical Assistant.

Leveraging state-of-the-art Retrieval-Augmented Generation (RAG) and Vision-Language Models (VLMs), this application will ingest user queries (both text descriptions and medical images, such as X-rays or dermatological photos). It will cross-reference these inputs against a custom Vector Database of authoritative medical textbooks to provide simplified, accurate explanations. Furthermore, the system integrates a structured data pipeline to act as a "Matchmaker," connecting the user's identified symptoms to top-rated local specialists using a custom recommendation algorithm. The entire system will be orchestrated via LangGraph and deployed via a FastAPI backend and a Streamlit frontend.

2. Project Objectives

The primary goal of this project is to build a production-ready, full-stack AI application. The specific technical and functional objectives include:

- **Unstructured Data Ingestion (RAG):** Successfully parse, chunk, and dynamically tag complex medical PDFs (e.g., Radiology and Symptom guides) using Docling and local Large Language Models (Llama 3.2), storing them in a Vector Database (ChromaDB/Pinecone).
- **Multimodal Diagnostic Capabilities:** Integrate a Vision LLM capable of accurately interpreting medical imagery (e.g., skin rashes, basic X-rays) and converting them into actionable text descriptions for the RAG pipeline.
- **Intelligent Routing:** Develop a robust LangGraph state machine to autonomously route user requests (casual chat vs. medical query vs. image upload) to the appropriate tools and functions.
- **Data-Driven Recommendations:** Clean and process real-world structured datasets (doctor ratings, locations, specialties) into an SQL database, and build an algorithm to map AI-generated symptoms to the top 3 highest-rated local specialists.
- **Scalable Full-Stack Deployment:** Wrap the AI logic in a secure REST API (FastAPI) and deliver it through an interactive, state-managed user interface (Streamlit).

3. Project Scope

To ensure the project remains focused and achievable for a two-person engineering team, the boundaries of the project are defined as follows:

In-Scope:

- **Data Engineering:** Collection, Exploratory Data Analysis (EDA), and cleaning of both unstructured (PDF textbooks) and structured (CSV/JSON doctor datasets) medical data.
- **AI Architecture:** Development of a Hybrid Search vector database (vector similarity + BM25 keyword matching) and tool-calling capabilities for the LLM.
- **Backend Infrastructure:** Creation of an SQL database for user chat history and doctor profiles, alongside a FastAPI server to act as the central bridge.
- **Frontend UI:** Development of a conversational Streamlit interface with image-upload capabilities and custom UI cards for doctor recommendations.
- **Telemetry:** Basic logging of Vector DB response times, LLM function calls, and error tracking.

Out-of-Scope:

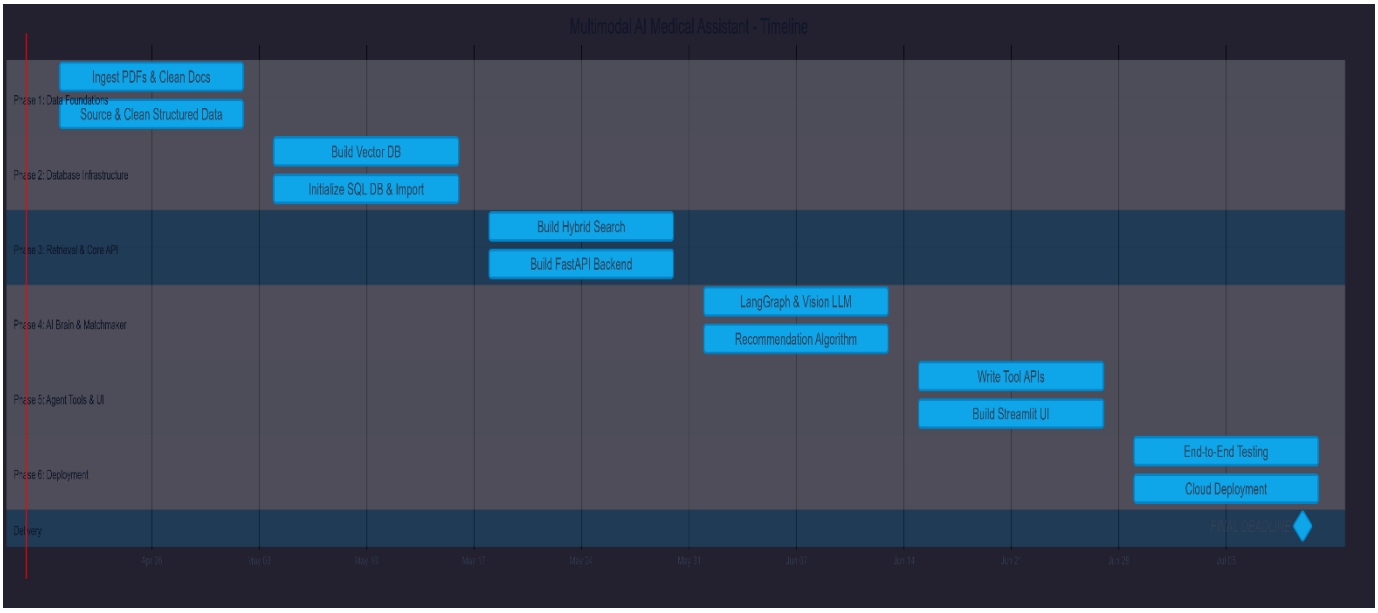
- **Official Medical Diagnosis:** The system is an educational and recommendation tool, not a certified medical device. It will not replace professional medical advice, and strict safety disclaimers will be hardcoded into the UI.
- **Direct Appointment Booking:** While the system will recommend doctors, it will not integrate with live hospital booking APIs or process payments/insurance claims.
- **HIPAA Compliance:** As a portfolio/academic project, enterprise-grade HIPAA compliance and heavy encryption protocols for patient data are not required for this phase.

2.1 Timeline & Gantt Chart Breakdown

The project execution spans 12 weeks, concluding with the final delivery on **July 10, 2026**.

Week (Sprint)	Dates	Phase Focus	Person A (AI / Data) Tasks	Person B (Cloud / UI) Tasks	Status
Week 1-2	Apr 20 - May 1	**Phase 1: Data Foundations**	Ingest PDFs, chunk data, tag metadata using Llama 3.2.	Source structured datasets, perform EDA, map taxonomy.	Planned
Week 3-4	May 4 - May 15	**Phase 2: Database Infrastructure**	Embed chunks and build the ChromaDB/Pinecone Vector DB.	Initialize PostgreSQL/MySQL; import clean structured data.	Planned
Week 5-6	May 18 - May 29	**Phase 3: Retrieval & Core API**	Build Hybrid Search logic; test Retrieval accuracy.	Build FastAPI skeleton; build endpoints for the Vector and SQL DBs.	Planned
Week 7-8	Jun 1 - Jun 12	**Phase 4: The AI "Brain" & Matchmaker**	Integrate Vision LLM; map out the LangGraph state machine.	Build telemetry logging; integrate the Recommendation scoring logic into the API.	Planned
Week 9-10	Jun 15 - Jun 26	**Phase 5: Agent Tools & Frontend**	Write Tool APIs (search DB, fetch doctors); refine system prompts.	Build Streamlit UI (chat state, file uploader, doctor recommendation cards).	Planned
Week 11-12	Jun 29 - Jul 10	**Phase 6: Deployment & Handover**	End-to-end edge-case testing; finalize model evaluation.	Containerize (Docker); deploy to cloud host; finalize documentation.	Planned
DEADLINE	**July 10, 2026**	**Project Delivery**	**Final System Turn-In**		

2.2 Gantt Chart Representation



2.3 Key Milestones & Deliverables

- **Milestone 1: The Data Freeze (May 15)**
  - *Deliverable:* All medical PDFs are successfully parsed into the Vector DB, and all real-world doctor data is cleaned and loaded into the SQL DB. Data gathering is officially closed.
- **Milestone 2: The Logic Bridge (May 29)**
  - *Deliverable:* The FastAPI backend successfully connects to both databases. Manual API queries return correct medical chunks and correct doctor lists.
- **Milestone 3: The Brain Activation (June 12)**
  - *Deliverable:* LangGraph is fully operational. The system can autonomously look at a medical image, generate text, and decide which database to search without human intervention.
- **Milestone 4: Minimum Viable Product (MVP) (June 26)**
  - *Deliverable:* The Streamlit UI is live on a local machine. A user can upload an image, chat with the bot, and receive UI cards showing recommended specialists.
- **Milestone 5: Production Launch (July 10)**
  - *Deliverable:* The application is fully deployed to the cloud, containerized, bug-free, and accessible via a public web link. Final documentation is submitted.

3.1 Defined Responsibilities by Team Member

1. Mohamed Atef Mohamed

- **Vector Database Management:** Responsible for embedding tagged medical chunks into a persistent vector database (Pinecone).
- **LangGraph Construction:** Architecting the core state machine that acts as the "Brain," routing user requests (chatting vs. medical queries vs. image uploads).
- **Vision LLM Integration:** Setting up the multimodal endpoint to parse user images (e.g., X-rays, skin rashes) and convert them into descriptive text.

2. Salma Fahmy Hassan

- **Data Engineer (Structured Data):** Owning the structured data pipeline, including data sourcing, Exploratory Data Analysis (EDA), and pipeline delivery for doctor/specialty datasets.
- **Retrieval Optimization:** Building and refining the Hybrid Search functions (combining vector similarity and BM25 keyword matching) to ensure the LLM pulls the most accurate medical context.

3. Rahma Ramadan

- **Data Engineer (Unstructured Data):** Developing the document ingestion pipeline to parse, clean, and format raw medical textbook PDFs.
- **Streamlit Application:** Leading frontend development by building the interactive UI, managing the chat window state, image uploader, and designing custom UI cards for doctor recommendations.

4. Ahmed Samy Essa

- **Data Engineer (Unstructured Data):** Collaborating on the document ingestion and PDF processing pipeline alongside Rahma.
- **Function Calling / Tool API:** Writing and securing the backend tools that the LLM triggers autonomously, such as `search_medical_database`.

5. Marihan Emad Eldeen

- **Vision LLM Integration:** Collaborating with Mohamed on the multimodal image-parsing capabilities.
- **Function Calling / Tool API :** Working with Ahmed to develop and deploy the autonomous tool-calling logic (e.g., `fetch_doctor_recommendations`).

6. Ali Mohamed Soliman

- **The Recommendation System:** Building the symptom-to-specialty mapping taxonomy that takes the LLM's medical output (e.g., "Eczema") and dynamically queries the database for the correct doctor specialty.
- **Telemetry & Logging:** Implementing backend tracking to monitor Vector DB response times, catch Vision model errors, and securely log API function calls.

3.2 Component Ownership and Team Responsibilities

A quick-reference guide for individual component ownership:

Phase / Task	Primary Owner(s)
1. Structured Data (EDA & Pipeline)	Salma Fahmy Hassan
2. Unstructured Data (Document Ingestion)	Rahma Ramadan, Ahmed Samy Essa
3. Vector DB Management (Pinecone)	Mohamed Atef Mohamed
4. Retrieval Optimization (Hybrid Search)	Salma Fahmy Hassan
5. LangGraph Construction	Mohamed Atef Mohamed
6. Vision LLM Integration	Mohamed Atef Mohamed, Marihan Emad Eldeen
7. Function Calling & Tool APIs	Marihan Emad Eldeen, Ahmed Samy Essa
8. Recommendation System (Matchmaking)	Ali Mohamed Soliman
9. Telemetry & Logging	Ali Mohamed Soliman
10. Streamlit UI Application	Rahma Ramadan

---

4. Risk Assessment & Mitigation Plan

4.1 Overview

Building a Multimodal AI Medical Assistant introduces unique technical and ethical challenges. This section outlines the primary risks associated with Large Language Models (LLMs), unstructured data parsing, and system integration, along with the specific strategies the engineering team will implement to mitigate them.

4.2 Technical & AI Risks

Risk 1: LLM Medical Hallucinations (High Severity)

- **Description:** The Llama 3.2 model may generate plausible-sounding but medically inaccurate advice, or invent treatments that are not present in the medical textbooks.
- **Mitigation Strategy: * Strict RAG Grounding:** The system prompt will strictly instruct the LLM to *only* use context retrieved from the Pinecone Vector Database. If the answer is not in the context, the LLM will be programmed to respond: *"I do not have enough information to answer this safely."*
    - **Hardcoded Disclaimers:** The Streamlit UI will feature un-closable banners stating the tool is for educational purposes only and not a substitute for professional medical diagnosis.

Risk 2: Vision Model Misinterpretation (High Severity)

- **Description:** The Vision LLM fails to accurately parse a user's uploaded medical image (e.g., mistaking a normal skin blemish for a severe rash), leading to a flawed downstream database search.
- **Mitigation Strategy:**
    - **Human-in-the-Loop Confirmation:** Before searching the database, the Vision LLM will output a text summary of what it *thinks* it sees in the image. The Streamlit UI will require the user to click "Confirm" or "Edit" on this text description before the LangGraph agent proceeds to the next step.

Risk 3: Poor Retrieval Accuracy from Vector DB (Medium Severity)

- **Description:** The AI fails to pull the correct medical chunks because the user's query uses different terminology than the textbooks (e.g., user types "stomach ache," but the book says "abdominal distress").
- **Mitigation Strategy:**
    - **Hybrid Search Implementation:** The Retrieval Engineer (Salma) will implement Hybrid Search. This combines standard vector similarity (which

understands semantic meaning) with BM25 keyword matching to ensure maximum retrieval accuracy.

Risk 4: PDF Parsing & Chunking Failures (Medium Severity)

- **Description:** Docling struggles to accurately parse complex medical tables or multi-column layouts from the raw textbook PDFs, resulting in messy, unreadable data chunks.
- **Mitigation Strategy:**
    - **Aggressive EDA:** The Unstructured Data team (Rahma and Ahmed) will perform extensive Exploratory Data Analysis on the output text before embedding it. If tables fail to parse, they will adjust Docling's OCR settings or manually exclude overly complex visual diagrams from the ingestion pipeline.

4.3 Operational & Integration Risks

Risk 5: Frontend/Backend Integration Bottlenecks (Medium Severity)

- **Mitigation Strategy:**
    - **API Contracts (Swagger/OpenAPI):** During Week 3, the backend team will publish a strict "API Contract" with dummy endpoints. This allows the frontend team to build the entire Streamlit UI using fake data while the backend team finishes the complex LangGraph logic.

---

4. 5. Key Performance Indicators (KPIs)

5.1 Overview

To objectively measure the success of the End-to-End Multimodal AI Medical Assistant, the engineering team has established strict Key Performance Indicators (KPIs). These metrics will be actively monitored via our backend telemetry systems to ensure the application is fast, accurate, and highly reliable prior to the July 10 delivery date.

5.2 AI & Retrieval Metrics (The "Brain")

These metrics measure the intelligence and accuracy of the underlying machine learning models and the Pinecone vector database.

- **Vector Retrieval Latency:** The time it takes for the Hybrid Search algorithm to query Pinecone and return the top medical chunks.
  - *Target Goal:* Under 1.5 seconds per query.
- **Context Relevance Score (RAG Hit Rate):** The percentage of user queries where the retrieved chunks directly contain the information needed to answer the question, minimizing hallucinations.
  - *Target Goal:* 90%+ contextual hit rate on test datasets.
- **Vision LLM Processing Time:** The total time required to ingest a user's uploaded medical image (e.g., an X-ray) and output a clean text description for the LangGraph agent.
  - *Target Goal:* Under 8 seconds per image.

5.3 System & Infrastructure Metrics (The "Body")

These metrics measure the stability and speed of the FastAPI backend and SQL databases.

- **End-to-End System Latency:** The total round-trip time from when a user hits "Send" in the Streamlit UI to when the AI's final text response begins streaming on the screen.
  - *Target Goal:* Under 4 seconds for pure text queries; under 12 seconds for multimodal (image) queries.
- **System Uptime & Stability:** The percentage of time the deployed web application and APIs are live and functional without crashing.
  - *Target Goal:* 99.9% uptime during the Phase 6 testing and deployment windows.
- **API Error Rate:** The percentage of backend function calls (e.g., `fetch_doctor_recommendations`) that fail due to timeout or bad data.
  - *Target Goal:* Less than 2% error rate in production.

5.4 Matchmaking & User Experience Metrics (The "Face")

These metrics measure how effectively the system serves the end-user and the accuracy of the recommendation engine.

- **Recommendation Accuracy:** The percentage of times the semantic taxonomy algorithm correctly matches the AI's predicted symptom to the exact appropriate medical specialty in the SQL database.
    - *Target Goal:* 95%+ routing accuracy.
- **End-User Task Success Rate:** The percentage of test users who are able to successfully upload an image, ask a question, and receive a doctor recommendation without abandoning the interface.
    - *Target Goal:* 85%+ completion rate during user testing.
- **Doctor Profile Render Speed:** The time it takes for the Streamlit UI to fetch data from the PostgreSQL/MySQL database and render the custom recommendation cards.
    - *Target Goal:* Under 1 second.